

RWTH Aachen University

M.Sc. Data Analytics & Decision Science

Vodafone Group Plc



Project Report

Pricing with Learning of Demand Curves under Inventory Constraint

Author and matriculation number:

Gurusaran Sivakumar – 469304

Praveen Balakrishnan - 467394

Shivam Patil - 471860

Vijayan Shinde - 465577

August 11th, 2025

Advisors:

Prof. Dr. Sven Müller

Chair of Data and Business Analytics

RWTH Aachen University

Dr. Guenter Klas

Senior Manager R&D, Research Clusters AI and Quantum

Vodafone Group Plc

Abstract

Dynamic pricing under inventory constraints is a critical problem in revenue management, especially when customer demand functions are unknown and heterogeneous. This project implements and tests the Primal-Dual Learning Algorithm for Personalized Dynamic Pricing with an Inventory Constraint, as proposed by Chen and Gallego (2018). The algorithm leverages the estimation of a dual variable representing the shadow price of inventory to decouple a multi-type pricing problem into independent single-type learning tasks, thereby avoiding the curse of dimensionality.

The implementation was developed in Python within a modular Jupyter Notebook framework, incorporating both the exploration phase (price discretization, demand observation, and dual estimation) and the exploitation phase (price interval refinement and inventory-aware allocation). Simulated demand data for multiple customer types were generated using logistic models and extended with machine learning estimators to capture unknown demand relationships. Key performance metrics including optimal prices, estimated demand functions, actual revenue, oracle benchmark revenue, and regret were computed and analyzed.

Experimental results demonstrate that the implemented algorithm consistently achieves low regret (all below 20, with a minimum near 5) and produces revenue close to the oracle benchmark across varying inventory scenarios. The findings validate the theoretical guarantees of the primal-dual approach and highlight its potential for scalable, personalized pricing strategies in practice.

Acknowledgements

We would like to express our sincere gratitude to Dr. Guenter Klas, Dr. Simone Mangiante, and Ana Ccarita from Vodafone for providing the opportunity to undertake this project and for their continuous support, valuable insights, and encouragement throughout its development. We also wish to extend our deep appreciation to professor Dr. Sven Müller for facilitating this collaboration and for offering consistent guidance, constructive feedback, and expertise during all stages of the project.

Contents

Main topic No.	Topic	Pg no.
	List of figures	05
	List of codes	06
	List of flowcharts	07
1.0	Introduction	08
	1.1 Motivation and industry context	08
	1.2 Problem statement	09
	1.3 Proposed solution: primal-dual learning framework as the core approach	12
	1.4 Project Objectives	14
2.0	Literature Review	18
3.0	Methodology	20
	3.1 Problem formulation	20
	3.2 Primal dual learning algorithm:	30
4.0	Implementation:	36
	4.1 Code structure and implementation flow:	36
	4.2 Dual Variable Estimation	37
	4.2 Demand Generation:	40
	4.3 Demand estimation function:	43
5.0	Results	51
6.0	Abbreviations:	57
7.0	References	58

List of figures:

Figure no.	Title	Pg no.
1	Representation of lagrangian formulation	23
2	logistic S shaped curve,	42
3	Random forest representation image.	47
4	Price vs demand in phase 1 for customer type 1	51
5	Price vs demand in phase 1 for customer type 2	52
6	Price vs demand after all phases for customer type 1	52
7	Price vs revenue after all phases for customer type 1	53
8	Price vs demand after all phases for customer type 2	53
9	Price vs revenue after all phases for customer type 2	54

List of codes:

Code no.	Title	Pg no.
Code 1 :	Dual Variable	40
Code 2:	Demand function	43
Code 3 :	Random Forest function	50

List of flowcharts:

Chart no.	Title	Pg no.
1 :	Phase 1 of algorithm (Exploration)	30
2:	Phase 2 of algorithm (Exploitation)	33

1. Introduction

1.1 Motivation and industry context

The telecommunications sector is marked by persistent margin compression arising from high operating expenditures, continuous capital investments in next-generation networks, and intensified competition in the B2C market. Over-the-top (OTT) applications (e.g., Netflix, YouTube, Spotify, WhatsApp) leverage mobile networks to deliver bandwidth-intensive services without directly compensating operators, structurally shifting value away from network providers. In response, mobile network operators (MNOs) are experimenting with new revenue models, including value-added services, private 5G networks, and Network-as-a-Service (NaaS), yet the commercial upside remains uncertain relative to the scale of investment required.

Within this strategic search, a novel proposition has emerged: monetizing excess compute capacity within the radio access network (RAN). Because RANs are provisioned for peak load, compute—especially GPU accelerators—tends to be underutilized during off-peak periods. Concurrently, operators are injecting artificial intelligence (AI) into 5G/6G RAN operations (e.g., energy optimization, adaptive beam steering, autonomous control), a trend that benefits materially from GPU acceleration. This convergence suggests a potential two-sided opportunity: leverage GPUs for internal AI operations during peak windows and commercialize idle capacity for external enterprise workloads during non-peak windows, as various industry vendors have begun to discuss.

Despite vendor enthusiasm, operators remain cautious. The perceived risks include operational complexity, the difficulty of balancing internal and third-party demands, uncertain external willingness to pay, and the challenge of designing pricing and allocation mechanisms that reliably produce net revenue gains after accounting for additional costs and risks. Revenue management is therefore a pivotal unresolved element in assessing the commercial viability of RAN compute monetization.

1.1.1 Research gap and rationale

Existing discourse emphasizes technological feasibility and supply-side enablement (AI-ready hardware, virtualization, cloud-native RAN). However, there is limited rigorous analysis of how to price and allocate scarce, time-varying GPU inventory across heterogeneous products and uncertain demand while respecting the operator's primary mission of reliable network performance. This study addresses that gap by formulating and analyzing a revenue management problem tailored to the telco RAN context with AI-accelerated compute as the binding resource.

1.2 Problem statement

We study a mobile network operator that deploys GPU resources to support AI-enhanced 5G/6G RAN operations. GPU utilization is time-varying: it is high during daytime and telco peak hours and lower during off-peak periods, creating windows where excess capacity can be offered to enterprise customers. Compute is cloudified and virtualized, enabling productization and time-windowed access.

- The operator sells GPU capacity in discrete units up to a capacity limit (c) (resource constraint).
- Sales occur over a finite booking horizon ($t \in [0, T]$), with purchased capacity scheduled for use in a subsequent service window ($t \in [T, 2T]$); revenue is realized over the booking horizon.
- Two product types are offered:
 - **Reserved instances:** exclusive, non-interruptible access during operator-specified hours in the service window.
 - **Preemptable instances:** lower-priced access that may be interrupted intermittently, suitable for non-critical workloads.
- All instances share the same GPU hardware type; GPUs are the binding resource and are economically more valuable than CPUs.

- The operator observes its internal time-varying GPU utilization and knows the windows in which capacity can be safely offered to third parties.

The operator's objective is to design a pricing and allocation policy that maximizes expected revenue from selling idle GPU units during the booking horizon, subject to stochastic demand and inventory constraints across products and time windows. The policy must determine product-specific prices and acceptance decisions over time while ensuring that allocations do not compromise internal network performance in the service window.

Formally, this is a finite-horizon revenue management problem with:

- **Inventory constraint** (c) on GPU units available for external sale;
- **Time coupling** between booking and service windows $([0, T])$ and $([T, 2T])$;
- **Product differentiation** via reserved versus preemptable instances;
- **Uncertain, potentially nonstationary demand** for each product type as a function of price;
- **Operational feasibility constraints** reflecting the primacy of internal telco workloads.

1.2.1 Scope and assumptions

- **Cloudified compute:** RAN-adjacent AI workloads run on virtualized cloud infrastructure; GPU resources are fungible across instances.
- **Internal priority:** Internal AI workloads have priority during peak hours; only off-peak windows are commercialized for third-party use.
- **Capacity granularity:** External capacity is sold in discrete GPU units up to (c) .
- **Booking-service separation:** Sales occur in $([0, T])$; service is delivered in $([T, 2T])$ with revenue attributed to $([0, T])$.
- **Product catalogue:** Only two products—reserved and preemptable—powered by identical GPU hardware; preemptable instances may be interrupted for short periods.

- **Information:** The operator knows internal utilization profiles sufficiently to publish time windows and quantities for external sale.

1.2.2 Objectives and research questions

- **Primary objective:** Maximize expected revenue from idle GPU capacity while preserving internal network performance guarantees.
- **Research questions:**
 1. How should prices for reserved and preemptable instances be set over the booking horizon under demand uncertainty.
 2. How should limited GPU inventory be allocated across products and time windows to balance yield and risk.
 3. What is the revenue impact of pre-emption policies and internal demand variability on feasible external offerings.
 4. Which revenue management approach (e.g., dynamic pricing with inventory shadow values) achieves robust performance with tractable complexity for operators.

1.2.3 Expected contribution

- A formal revenue management formulation for RAN compute monetization with booking–service separation and product differentiation.
- A pricing and allocation framework that incorporates inventory constraints, demand uncertainty, and pre-emption risk.
- Quantitative evaluation via simulation under realistic utilization patterns, demonstrating trade-offs between reserved and preemptable mix, pricing levels, and capacity limits (c).

1.3 Proposed solution: primal–dual learning framework as the core approach

This report proposes adopting the **Primal–Dual Learning Algorithm for Personalized Dynamic Pricing with an Inventory Constraint**, introduced by Chen and Gallego (2018), as the central decision-making and optimisation engine for addressing the project’s challenges. This algorithm directly targets the two most pressing issues in our context:

1. **Limited, high-value capacity** that must be allocated intelligently across offerings.
2. **Unknown and variable demand** patterns that need to be learned and acted upon quickly.

1.3.1 Why primal dual learning method is a strong fit

- **Strategic simplicity with computational efficiency**

Instead of wrestling with the complexity of managing multiple customer segments and pricing strategies all at once, the primal–dual approach reframes the problem so that each segment’s pricing decision can be optimised independently. The segments are coordinated through a single unifying factor — the estimated economic value of the constrained resource. This design keeps the computational requirements low, even when the number of product types is large.

- **Adaptive learning for uncertain environments**

Demand behaviour for different customer types is not assumed to be known in advance. The algorithm methodically tests different prices, observes customer responses, and adjusts its estimates of both demand and the “shadow price” of capacity. This allows it to adapt in real time to emerging market conditions.

- **Scalable to multiple offerings**

Whether there are two distinct product tiers or a dozen, the algorithm's performance does not deteriorate with added complexity. This scalability is essential for environments where the product mix may evolve or expand.

- **Proven performance and theoretical guarantees**

Chen and Gallego demonstrated that their approach maintains low performance loss compared to an all-knowing "oracle" policy, and critically, that this loss does not grow with the number of customer types. That means reliable results without runaway complexity.

1.3.2 How primal dual learning works in practice

The method unfolds in two clear stages:

1. **Exploration** – Systematically vary prices for each customer type to collect demand data. This is a structured process, not random trial and error, ensuring that the right information is gathered quickly.
2. **Exploitation** – With the data in hand, focus on prices that maximise revenue, while staying within the capacity limit. Adjustments are made if the inventory runs low faster than expected, helping avoid stock-outs or suboptimal allocation.

1.3.3 Relevance to the project scenario

In our case — managing and monetising a finite GPU pool across multiple offerings — the parallels are exact:

- The **capacity limit** is the fixed number of GPUs available for sale at any given time.
- The **multiple customer types** are the different instance types or market segments with distinct demand curves.

- The **unknown demand** is the varying willingness to pay for GPU time across these segments.
- The **unifying variable** is the estimated marginal value of the GPUs, which guides pricing across all segments at once.

By using Chen & Gallego's primal–dual learning framework, the project gains a proven, academically grounded method that can guide decisions from day one, adapt to real-world demand behaviour, and ensure every GPU hour is sold to the right customer at the right price.

1.4 Project Objectives

This project is motivated by the need to develop a scalable and adaptive pricing strategy for monetizing excess GPU capacity in a resource-constrained telecommunications environment. Drawing on the primal–dual learning framework proposed by Chen and Gallego (2018), the project sets out to achieve the following core objectives:

1.4.1 Implement the Primal–Dual Learning Algorithm

The first objective is to translate the theoretical algorithm into a working implementation. The primal–dual learning algorithm is designed to solve dynamic pricing problems under inventory constraints by estimating both demand functions and the shadow price of inventory. In this project, the algorithm has been implemented in Python using a modular Jupyter Notebook structure. The implementation includes:

- A two-phase learning strategy: exploration followed by exploitation.
- Price discretization and demand estimation for each customer type.
- Calculation of the dual variable (shadow price) to guide pricing decisions.
- Interval refinement and adaptive pricing based on observed demand.

This implementation serves as a practical validation of the algorithm’s structure and logic, enabling controlled experimentation with simulated customer behavior.

1.4.2 Achieve Near-Optimal Regret

A key performance metric in learning-based pricing algorithms is **regret** — the difference between the revenue achieved by the algorithm and the revenue that would have been earned by an oracle with perfect knowledge of demand. The goal is to keep regret as low as possible, ideally achieving sublinear growth with respect to the time horizon.

In line with the theoretical guarantees provided in the paper, the implemented algorithm is evaluated for its ability to achieve **near-optimal regret**, independent of the number of customer types. This involves:

- Comparing actual revenue to oracle revenue across multiple runs.
- Measuring regret separately for reserved and preemptable customer types.
- Validating that regret remains within acceptable bounds as defined by the scaling regime.

1.4.3 Test with Simulated Multi-Type Customer Data

To assess the algorithm’s adaptability and scalability, the third objective is to test it using **simulated data representing multiple customer types**. Each type exhibits distinct demand behaviour and price sensitivity, reflecting realistic market segmentation. The simulation includes:

- Logistic and Poisson-based demand models.
- Reserved and preemptable instance types with different pricing dynamics.
- Time-varying inventory availability and booking windows.

This setup allows for a comprehensive evaluation of how well the algorithm handles personalization, inventory allocation, and dynamic pricing across heterogeneous

customer segments. It also provides insight into how the dual variable coordinates pricing decisions across types, ensuring efficient use of limited GPU resources.

1.4.4 Validate Algorithm Robustness Across Demand Variability

Beyond achieving low regret, a critical objective is to test the **robustness** of the algorithm under varying demand conditions. Real-world markets are rarely stationary, so the algorithm's ability to adapt to fluctuations in customer behavior is essential. This includes:

- Running simulations with noisy or non-stationary demand curves.
- Stress-testing the algorithm under extreme inventory constraints.
- Evaluating performance when customer preferences shift mid-horizon.

1.4.5 Analyze Dual Variable Behavior and Economic Interpretation

The dual variable (shadow price) plays a central role in coordinating pricing decisions. An additional objective is to **analyze its behavior** and interpret its economic significance. This involves:

- Tracking how the dual variable evolves over time.
- Understanding its impact on pricing decisions for different customer types.
- Relating it to opportunity cost and marginal value of inventory.

1.4.6 Explore Scalability and Computational Efficiency

As the number of customer types or pricing intervals increases, computational complexity can become a bottleneck. A forward-looking objective is to **explore the scalability** of the algorithm. You might include:

- Benchmarking runtime performance across different problem sizes.
- Identifying bottlenecks in demand estimation or interval refinement.
- Proposing optimizations or parallelization strategies.

1.4.7 Lay Groundwork for Real-World Deployment

Finally, if your project is intended to inform practical pricing strategies (e.g., for GPU resource allocation), you could include an objective focused on **real-world applicability**. This might involve:

- Designing a framework for integrating the algorithm into existing pricing systems.
- Identifying data requirements for live deployment.
- Discussing regulatory or ethical considerations in dynamic pricing.

2. Literature Review

Dynamic pricing, also referred to as revenue management pricing, is a strategic approach in which prices are adjusted over time in response to changes in demand, supply, and market conditions. Originating in the airline and hospitality industries (Talluri & van Ryzin, 2004), dynamic pricing has since been widely adopted across sectors such as e-commerce, ride-hailing, and telecommunications, where heterogeneous customer preferences and uncertain demand present significant challenges. Traditional models often assume prior knowledge of demand functions, enabling optimal prices to be computed directly. However, in many real-world settings, demand is initially unknown and must be learned from observed data while simultaneously managing operational constraints such as limited inventory.

A key challenge in such problems is the **exploration–exploitation trade-off**, where the decision-maker must balance learning about demand (exploration) with applying current knowledge to maximize revenue (exploitation). Classical approaches to this problem include stochastic approximation methods, multi-armed bandit algorithms (Bubeck & Cesa-Bianchi, 2012), and reinforcement learning-based strategies (Sutton & Barto, 2018). While these methods have achieved success in unconstrained or low-dimensional settings, their performance can degrade significantly when the number of customer types or pricing dimensions increases—a phenomenon often referred to as the “curse of dimensionality.”

The **Primal-Dual Learning Algorithm** proposed by Chen and Gallego (2018) addresses this scalability issue by leveraging the **dual formulation** of the constrained optimization problem. The algorithm estimates the **dual optimal variable**, also known as the shadow price of inventory, which represents the marginal value of an additional unit of stock. Once this dual variable is learned, the complex joint optimization problem over multiple customer types can be decomposed into independent single-type pricing subproblems. This decomposition not only reduces computational complexity but also ensures that the regret bound is independent of the number of customer types, a significant improvement over prior methods where regret typically scales with problem dimensionality.

In the primal formulation, the objective is to maximize expected revenue subject to inventory constraints, while the dual formulation introduces a Lagrange multiplier for the inventory constraint. The Chen & Gallego approach integrates learning into this framework by first running an **exploration phase** with discretized prices to estimate demand functions and the shadow price, followed by an **exploitation phase** where prices are refined based on these estimates. Their analysis demonstrates that the algorithm achieves a regret rate of $O(T^{-1/2})$ in the scaling regime, independent of the number of customer types, where T is the time horizon.

Subsequent studies have extended primal-dual and bandit-based methods to various domains, including online advertising (Balseiro et al., 2015), cloud computing resource allocation (Wang et al., 2020), and large-scale retail pricing (Ferreira et al., 2018). These works reinforce the value of dual-based approaches in managing resource-constrained optimization problems under uncertainty. Furthermore, machine learning models such as logistic regression, random forests, and gradient boosting have been incorporated into demand estimation, enabling more flexible representations of customer behavior beyond parametric demand functions.

This project builds directly on the Chen & Gallego framework, implementing the full primal-dual learning algorithm for a multi-type customer setting with simulated demand. The implementation incorporates both logistic demand models and machine learning estimators, allowing a comparison between theoretical and data-driven demand estimation techniques. By integrating these approaches, the work demonstrates the adaptability of the primal-dual learning methodology to practical scenarios where demand patterns are complex, dynamic, and initially unknown.

3. Methodology

3.1 Problem formulation:

3.1.1 Primal Problem:

The primal problem is designed to help a provider maximize revenue from selling GPU-powered compute instances over a fixed time horizon. At each time step, a customer arrives — either a Reserved or Preemptive type — and the provider must decide what price to offer. The decision is made without knowing the customer's exact willingness to pay, but the provider can estimate the probability that the customer will accept a given price.

The core idea is to choose prices that maximize expected revenue, which is calculated as the product of the offered price and the likelihood that the customer will make a purchase. This expected revenue is summed across all time steps.

However, the provider faces a critical constraint: limited inventory. The total number of GPU units available is fixed, and the provider must ensure that the cumulative expected demand across all customers does not exceed this inventory. This creates a tension between setting high prices (which may reduce demand) and allocating resources efficiently across different customer types.

In essence, the primal problem captures the trade-off between:

- **Maximizing revenue** by setting prices that reflect customer willingness to pay.
- **Managing inventory** by controlling how many units are expected to be sold over time.

This formulation is widely used in operations research and revenue management, especially in settings where resources are scarce and customer behavior is uncertain. It lays the foundation for learning-based algorithms that adapt pricing strategies in real time.

Primal problem formulation:

Primal Problem:

Objective function: **maximize revenue**

Normally, revenue = price x units sold

Revenue in implementation = price x demand x total customer
approached

$$\max_{\{p_t\}} \sum_{t=1}^T \mathbb{E}[p_t \cdot D(x_t, p_t)]$$

Constraint: **Inventory**, total demand should not exceed inventory.

$$\sum_{t=1}^T \mathbb{E}[D(x_t, p_t)] \leq c$$

Here,

p_t : price set at time t

x_t : type of customer at time t (could be Reserved or Preemptive)

$D(x_t, p_t)$: probability that a type- x_t customer buys at price p_t

c : total inventory (e.g., 100 GPUs)

3.2.2 Lagrangian formulation for primal problem relaxation

The **Lagrangian** is a central construct in constrained optimization, used to reformulate problems that involve maximizing or minimizing an objective function subject to one or more constraints. In its classical form, a constrained optimization problem consists of an

objective function and a set of constraints — typically inequalities or equalities — that restrict the feasible region of solutions. Solving such problems directly can be challenging, especially when constraints are complex or when the feasible region is nontrivial.

To address this, the Lagrangian method introduces a new function — the Lagrangian — which combines the original objective function with the constraints, weighted by auxiliary variables known as **Lagrange multipliers** or **dual variables**. These dual variables serve as penalty coefficients: they quantify how much the objective function should be penalized when the constraints are violated. By embedding the constraints into the objective function, the Lagrangian transforms the original constrained problem into a form that can be tackled using unconstrained optimization techniques.

Mathematically, the Lagrangian is constructed by taking the objective function and subtracting the product of each constraint and its corresponding dual variable. The resulting function depends on both the original decision variables and the dual variables. The optimization then proceeds by finding saddle points of the Lagrangian — points where the function is minimized with respect to the primal variables and maximized with respect to the dual variables.

Application in the Paper: Inventory-Constrained Revenue Maximization

In the context of the paper, the primal problem involves maximizing expected revenue from selling compute instances over a finite time horizon, subject to an inventory constraint. The constraint ensures that the total expected demand does not exceed the available GPU units. This is a classic example of a resource-constrained optimization problem.

To solve this, the authors formulate the **Lagrangian** by incorporating the inventory constraint into the revenue objective. The dual variable introduced in this formulation represents the **shadow price of inventory** — that is, the marginal value of one additional unit of GPU capacity. When inventory is scarce, this value is high, and the Lagrangian penalizes pricing decisions that lead to excessive demand. When inventory is abundant, the penalty is low, allowing more aggressive pricing.

This reformulation enables the use of **primal-dual algorithms**, which iteratively update both the pricing decisions (primal variables) and the inventory valuation (dual variable). At each time step, the algorithm selects a price that maximizes the Lagrangian, given the current estimate of the dual variable. Simultaneously, it adjusts the dual variable based on observed demand and remaining inventory. This dynamic interplay allows the algorithm to learn both the demand function and the optimal pricing strategy over time, without requiring full prior knowledge of customer behavior.

Lagrangian Formulation example:

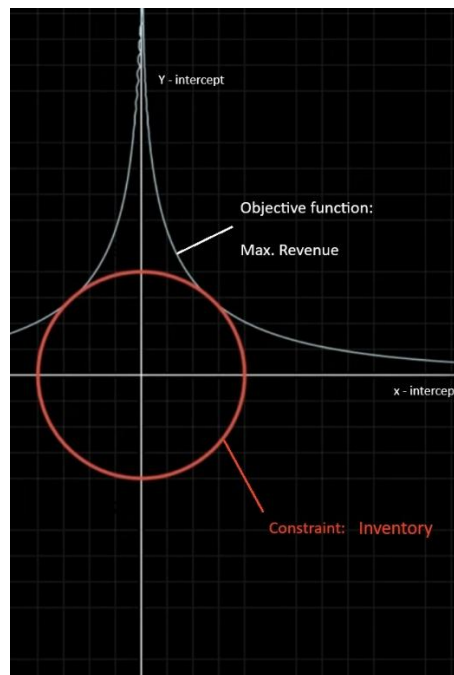


Figure .01: Representation of lagrangian formulation, hyperbola represents objective function (maximize revenue) and circle represents constraint (limited inventory).

We are given an objective function (Hyperbolic Curve in above figure – maximize revenue):

$$R(x, y) = x^2 e^y$$

subject to the constraint (red circle in above figure – limited inventory):

$$B(x, y) = x^2 + y^2 = 4$$

The necessary condition for optimality is that the gradients of the objective and constraint functions are proportional:

$$\nabla R = \lambda \nabla B$$

This condition ensures that the gradients (directions of steepest ascent) of the revenue and constraint functions are aligned—meaning, at a point of optimal trade-off.

To solve this constrained optimization problem using the Lagrangian method, we define the Lagrangian function:

$$L(x, y, \lambda) = R(x, y) - \lambda(B(x, y) - b)$$

Here,

λ : The Lagrange multiplier, representing how much the objective function would improve if the constraint were relaxed.

From Primal to Lagrangian Formulation

The transition from the primal problem to the Lagrangian formulation involves the following conceptual steps:

1. **Start with the primal problem:** Maximize expected revenue subject to an inventory constraint.
2. **Introduce a dual variable:** This variable represents the cost of violating the inventory constraint.
3. **Construct the Lagrangian:** Combine the revenue objective and the constraint, weighted by the dual variable.
4. **Optimize the Lagrangian:** Use iterative methods to find the best pricing decisions and update the dual variable based on feedback.

$$\mathcal{L}(\{p_t\}, z) = \sum_{t=1}^T \mathbb{E}[(p_t - z) \cdot D(x_t, p_t)] + z \cdot c$$

p_t : price set at time t

x_t : type of customer at time t (could be Reserved or Preemptive)

$D(x_t, p_t)$: probability that a type- x_t customer buys at price p_t

c : total inventory (e.g., 100 GPUs)

z : unit value of inventory.

This approach transforms a constrained optimization problem into a structure that is more amenable to learning and adaptation. It also provides interpretability: the dual variable gives insight into how valuable the inventory is at any point in time, guiding pricing decisions in a principled way.

3.2.3 Dual Problem Formulation

In constrained optimization, the dual problem provides a complementary perspective to the primal formulation. While the primal problem seeks to maximize expected revenue subject to an inventory constraint, the dual problem reframes this by introducing a penalty variable — the **dual variable** — that quantifies the value of the constraint itself. This dual variable, denoted (z) , represents the **unit value of inventory**, and plays a central role in coordinating pricing decisions across customer types.

Motivation for the Dual Problem

The primal problem is constrained by a fixed inventory capacity. Solving it directly requires balancing revenue against resource usage, which becomes complex when demand functions are unknown or vary across customer types. The dual problem simplifies this by embedding the constraint into the objective function via the Lagrangian. This allows the optimization to proceed in two stages:

1. **Maximize the Lagrangian over prices** for each customer type and time period.

2. **Minimize the resulting function over the dual variable (z),** which must remain non-negative.

This two-step structure enables the algorithm to learn both demand behavior and the economic value of inventory simultaneously.

Dual Objective Function

The dual problem is defined by the function:

$$g(z) = \sum_{t=1}^T \max_{p \in \mathcal{P}} (p - z) \cdot D(x_t, p) + z \cdot c$$

p_t : price set at time t

x_t : type of customer at time t (could be Reserved or Preemptive)

$D(x_t, p_t)$: probability that a type- x_t customer buys at price p_t

c : total inventory (e.g., 100 GPUs)

z : unit value of inventory.

This function captures the trade-off between pricing decisions and inventory valuation. The first term computes the maximum expected profit at each time step, adjusted by the dual variable. The second term adds the total value of the inventory, scaled by (z).

Algorithmic Interpretation

In the primal–dual learning algorithm, the dual problem guides the selection of prices by estimating how valuable each unit of inventory is. The algorithm proceeds as follows:

- For each customer arrival, it evaluates all allowable prices and selects the one that maximizes the adjusted profit
- It then updates the dual variable (z) based on cumulative demand and remaining inventory, minimizing the overall dual objective.

This iterative process ensures that pricing decisions are not made in isolation but are coordinated through a shared valuation of inventory. As inventory becomes scarce, (z) increases, leading to more conservative pricing. When inventory is abundant, (z) decreases, allowing more aggressive pricing.

Practical Role in Implementation

In the Jupyter Notebook implementation, this dual formulation is central to the algorithm's logic. You:

- Compute the optimal price for each customer type by maximizing the adjusted revenue term.
- Track and update the dual variable (z) based on observed demand and inventory usage.
- Use (z) to guide future pricing decisions, ensuring that inventory is allocated efficiently across the booking horizon.

This dual perspective not only simplifies the optimization but also provides interpretability: the dual variable acts as a dynamic signal of resource scarcity, shaping pricing behavior in real time.

3.2.4 Dual variable and optimal pricing:

The optimal dual variable (z^*) is the value of (z) that minimizes the dual objective function. It represents the **true marginal value of inventory** under the current demand environment and pricing structure. Once computed, (z^*) becomes the central coordinating signal for pricing decisions across all customer types and time periods.

Mathematical Definition

$$z^* = \arg \min_{z \geq 0} \left\{ \sum_{t=1}^T \max_{p \in \mathcal{P}_{x_t}} (p - z) \cdot \hat{D}_{x_t}(p) + z \cdot c \right\}$$

p_t : price set at time t

x_t : type of customer at time t (could be Reserved or Preemptive)

$D(x_t, p_t)$: probability that a type- x_t customer buys at price p_t

c : total inventory (e.g., 100 GPUs)

z : unit value of inventory.

- **Revenue generation** from each customer, adjusted by the penalty (z).
- **Inventory valuation**, represented by the term ($z \cdot c$).

The optimal (z^*) is the point where this trade-off is minimized — where the pricing strategy yields the highest adjusted revenue without over-consuming inventory.

Operational Role in the Algorithm

Once (z^*) is computed, it guides the pricing strategy as follows:

- For each arriving customer, the algorithm selects the price (p) that maximizes

$$p^*(x) = \arg \max_{p \in \mathcal{P}} (p - z^*) \cdot \hat{D}_x(p)$$

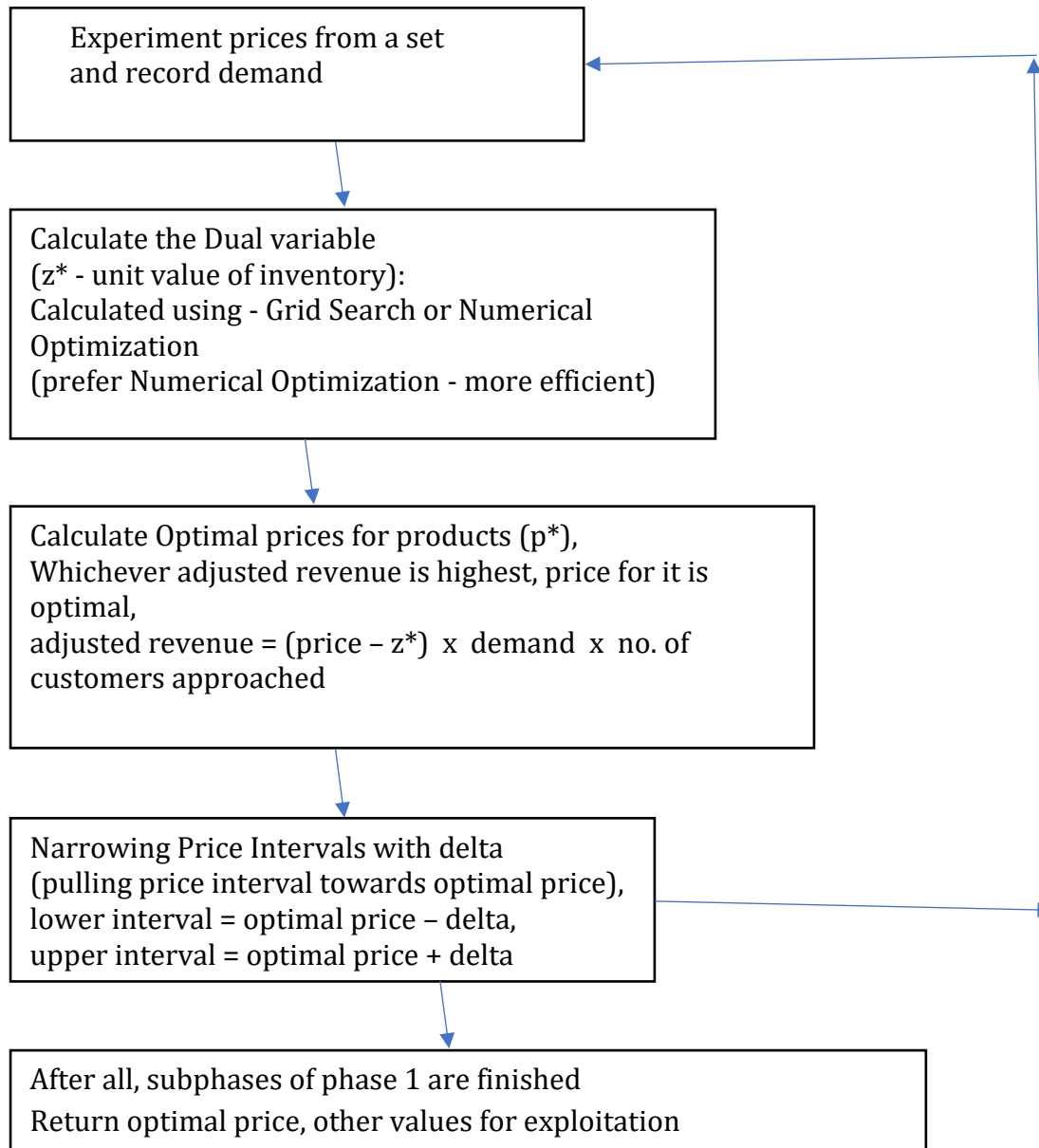
- This ensures that prices are chosen not just for immediate profit, but also in consideration of inventory scarcity.
- As demand patterns evolve, (z^*) is periodically re-estimated to reflect updated beliefs about customer behavior and remaining inventory.

Dynamic Behavior

- Early in the booking horizon, when inventory is plentiful, (z^*) tends to be low. This encourages exploration and demand learning.
- Later, as inventory depletes, (z^*) rises, shifting the algorithm toward exploitation and more conservative pricing.

This dynamic adjustment makes (z^*) a powerful control signal — it adapts to both temporal constraints and demand uncertainty, enabling robust revenue management.

3.2 Primal dual learning algorithm:



Flowchart 1: Phase 1 of algorithm (Exploration).

The **Primal-Dual Learning Algorithm**, as introduced by Chen and Gallego (2018), provides a robust framework for dynamic pricing under inventory constraints and demand uncertainty. Its relevance to the monetization of idle GPU capacity in telco RAN environments stems from its ability to simultaneously learn demand behavior and optimize pricing decisions in real time, while respecting resource limitations and operational priorities.

This section presents a detailed exposition of the algorithm's two-phase structure—**Phase I: Exploration** and **Phase II: Exploitation**—with emphasis on its mechanics,

decision logic, and application to differentiated product offerings (reserved and preemptable GPU instances).

3.2.1 Phase I: Exploration – Demand Learning and Shadow Price Estimation

This phase focuses on understanding how customers respond to different prices and estimating the value of inventory.

Step 1: Price Experimentation

For each product type (reserved and preemptable), the operator selects a set of prices within a defined interval. These prices are offered to customers over multiple booking instances.

Step 2: Demand Observation

At each price, the operator records how many customers were offered the product and how many accepted. This helps estimate the demand function — essentially, how demand varies with price.

Step 3: Adjusted Revenue Calculation

For each price, the operator calculates adjusted revenue. This is the actual revenue minus the cost of consuming inventory, which is estimated using a variable called “dual variable.” The price that yields the highest adjusted revenue is considered the best price for that subphase.

Step 4: Interval Refinement

Once the best price is identified, the operator defines a narrower interval around it. For example, if the best price is p^* , the new interval might be from $p^* - \delta$ to $p^* + \delta$. This refined interval is used in the next subphase to test prices more precisely.

Step 5: Iteration Across Subphases

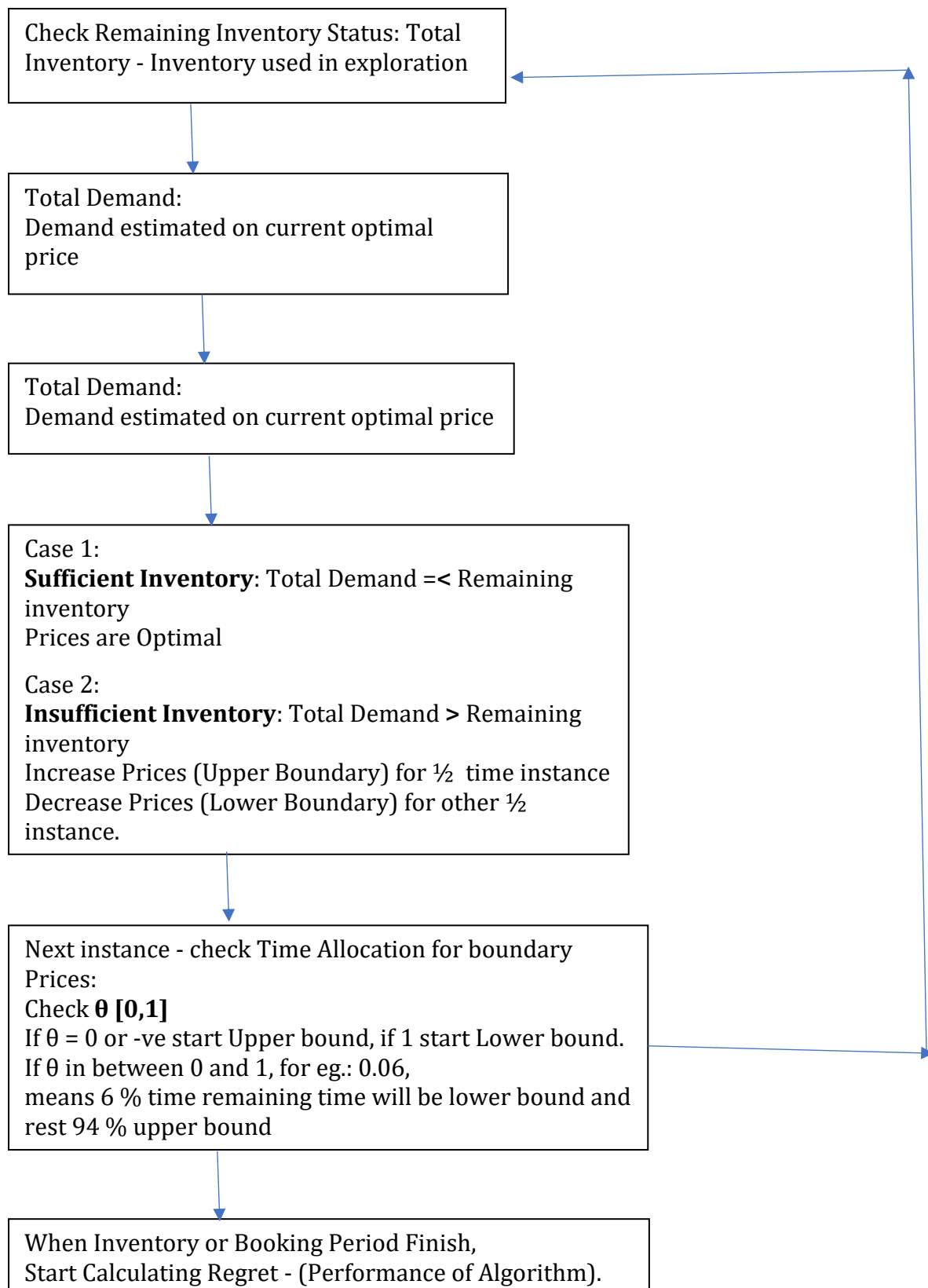
This process — testing prices, observing demand, calculating adjusted revenue, and refining the interval — is repeated across several subphases. Each iteration improves the accuracy of demand estimates and the valuation of inventory.

Step 6: Transition to Exploitation

At the end of exploration, the operator passes the following to the next phase:

- The estimated value of one unit of inventory (z^* , dual variable)

- The best price for each product type (p^* reserved, p^* preemptable)
- Remaining inventory (c)
- Remaining time in the booking horizon (T remaining)



Flowchart 2: Phase 2 of algorithm (Exploitation).

:

3.2.2 Phase II: Exploitation — Adaptive Pricing and Inventory Management

This phase uses the insights from exploration to make real-time pricing decisions that balance revenue and inventory.

Step 1: Demand Projection

Using the best price from exploration (p^*), the operator estimates daily demand. This is multiplied by the number of days left (T remaining) to project total future demand.

Step 2: Inventory Comparison

The projected demand is compared to the remaining inventory (c remaining). If inventory is sufficient, the operator continues offering the product at p^* . If not, prices are adjusted to slow down sales.

Step 3: Price Adjustment

To adjust pricing, the operator defines a new interval around p^* . For example:

- $p_{upper} = p^* + \delta$
- $p_{lower} = p^* - \delta$

In the next booking instance, the operator offers p_{upper} for the first half and p_{lower} for the second half.

Step 4: Test new prices (upper and lower bound on 1 instance):

Here, the algorithm experiments the upper and lower bound prices for 1 instance total, meaning half instance is for upper bound and other half is for lower bound. Again, the demand is recorded for both bounds of prices. This demand is used in next step for deciding which bound of price is more important.

Step 6: Time Allocation

Based on demand recorded in last step for halves of instances, first to upper bound of price and second to lower, the algorithm calculates θ , which tells us what percent of next two instances combine time should be allocated to upper and lower bound. If θ is 0.3 then 30% of next two instances time is allocated to lower bound and remaining 70% to upper. This ensures pricing reflects both demand sensitivity and inventory pressure.

Step 7: Iterative Refinement

This cycle — projecting demand, comparing inventory, adjusting prices, and reallocating time — continues until the booking horizon ends or inventory is depleted.

After completion of all phases, there is regret calculation which tells us about the different between actual revenue generated and optimal revenue. According to paper, if the regret is between 0 to 15%, it's assumed as a good regret.

Phase	Purpose	Key Variables Used	Key Actions
Exploration	Learn demand and inventory value	P^* , dual_variable, delta	Test prices, observe demand, refine intervals, estimate inventory value
Exploitation	Maximize revenue under constraints	p_upper, p_lower, c_remaining, T_remaining	Project demand, compare inventory, adjust prices, allocate time, iterate

Table : Summary of Algorithm Flow

Relevance to Telco RAN Monetization

This algorithm is ideal for telco RAN environments where:

- GPU inventory is limited and must be preserved for internal workloads
- Customer demand varies over time and across product types
- Pricing must adapt to real-time conditions
- Revenue optimization must be balanced with operational constraints

By combining learning and adaptive control, the Primal–Dual Learning Algorithm enables operators to monetize idle compute capacity intelligently, ensuring both profitability and service integrity.

4. Implementation:

4.1 Code structure and implementation flow:

- Take Input data from user.
- Run main loop with exploration function (phase 1) and user inputs as function attributes inputs till Phase 1 is complete (completing all subphases).
- In exploration for subphase iterations:
 - Run demand simulation, runs for all customers and prices in that subphase
 - Generate demand – buying probability
 - Record demand in a data frame with customer type and price.
 - Reduce inventory
 - Calculate dual variable – unit value of inventory
 - Calculate optimal price according to adjusted revenue
 - Narrow down price intervals according to alpha
 - Save data for plotting
- After ending phase, Plot price vs demand observed
- Go into second loop for Phase 2 (Exploitation function)
 - Run demand estimation taking optimal price value from phase 1
 - Compare demand vs inventory
 - Choose case sufficient inventory vs non-sufficient
 - In sufficient inventory conduct sales on current optimal price
 - In non-sufficient, change prices according to alpha to upper and lower bound
 - Record demand for one instance on both lower and upper bound of price
 - Calculate theta on the basis of demand recorded for an instance
 - According to theta allocate time and conduct sales on upper and lower bounds
 - Change optimal price to price (upper or lower bound) allocated to maximum time
- Record all data in final data frame.

- Make different data frames for data preparation for calculating results and visualization of results.
- Calculate results:
 - Calculate revenues
 - Calculate regret.
 - Plot recorded data according to price vs demand and price vs revenue.

4.2 Dual Variable Estimation

In constrained optimization, the **dual variable** plays a central role in coordinating decisions across competing objectives. In the context of telco RAN compute monetization, the dual variable — denoted as z — represents the **marginal value of inventory**, i.e., the economic worth of one additional unit of GPU capacity. Estimating this value accurately is essential for making informed pricing and allocation decisions under uncertainty.

4.2.1 Conceptual Role of the Dual Variable

The dual variable emerges from the Lagrangian formulation of the revenue management problem. It quantifies the trade-off between immediate revenue and long-term inventory preservation. Specifically:

- A **low value of z** indicates that inventory is abundant, allowing for more aggressive pricing.
- A **high value of z** signals scarcity, prompting the algorithm to raise prices and conserve inventory.

In your algorithm, z is used to adjust revenue calculations during both exploration and exploitation phases. It ensures that pricing decisions reflect not just customer demand, but also the opportunity cost of consuming limited resources.

4.2.2 Mathematical Formulation

The dual variable is estimated by solving the following optimization problem:

Find the value of z that minimizes the total adjusted cost across the booking horizon:

$$z^* = \arg \min_{z \geq 0} \left\{ \sum_{t=1}^T \max_{p \in \mathcal{P}_{x_t}} (p - z) \cdot \hat{D}_{x_t}(p) + z \cdot c \right\}$$

- p is the price offered at time t .
- x_t is the customer type at time t .
- c is the total inventory capacity.
- z is the dual variable.
- z^* is the optimal value of the dual variable.
- D is the demand for particular customer type at a price

This formulation balances two competing forces:

- The revenue generated from selling GPU units at various prices.
- The cost of consuming inventory, scaled by z .

4.2.3 Implementation via Numerical Optimization

In your code, the dual variable is estimated using **numerical optimization**, specifically the `minimize_scalar` method from SciPy. This approach is chosen for several reasons:

Why Numerical Optimization?

- **Nonlinear Objective:** The function being minimized involves a maximum over multiple price points and a non-parametric demand estimate. This makes analytical solutions impractical.
- **No Closed-Form Expression:** The demand function is learned via Random Forests and stored as a dictionary of probabilities, which cannot be differentiated symbolically.

- **Efficiency and Precision:** `minimize_scalar` efficiently searches for the optimal z within a bounded interval, ensuring convergence even when the function is irregular or noisy.

Code Logic

Your function `dual_objective` performs the following steps:

1. **Magnitude Adjustment:** Since the optimal z tends to be very small (e.g., 0.00001), you scale it by the order of magnitude of the optimal price to make it interpretable and comparable.
2. **Objective Definition:** For each candidate z , the function computes the maximum adjusted revenue across all prices, then adds the inventory cost ($z \times c$).
3. **Optimization:** The `minimize_scalar` method searches for the value of z that minimizes this objective.
4. **Output Scaling:** The final z^* is scaled and returned in a form that reflects its economic significance relative to price levels.

4.2.4 Strategic Importance

The dual variable z^* serves as a dynamic control signal throughout the algorithm:

- In **exploration**, it helps identify the price that maximizes adjusted revenue.
- In **exploitation**, it informs whether the current price should be maintained or adjusted based on inventory sufficiency.
- It ensures that pricing decisions are economically grounded, balancing short-term gains with long-term resource constraints.

By estimating z^* through numerical optimization, your algorithm remains flexible, data-driven, and robust to irregularities in demand behavior — a critical capability in real-world telecom environments.

Code:

```
#####  
#####  
#####  
# # Dual optimization: Find optimal value i.e. - z_opt in code and z* in paper  
for Exploration (P1)  
  
"""For bounds from 0 to maximum price"""  
""" Minimize_scalar used to optimize"""  
  
def dual_objective(z, demand_probability, p, c):  
  
    # Finding order of magnitude of optimal price respective to the customer  
    # Order of Magnitude is needed as the dual variable z* value comes in very  
small number e.g. 0.00000..irrespective of price e.g. 100 range or 1000 or  
10,000, etc...,  
    # and to make sense of dual variable we multiply it with magnitude of  
optimal price so it's with respect to optimal price and makes sense.  
    magnitude_p_opt = len(str(int(abs(min(p)))))  
    magnitude_order_p_opt=(10**(magnitude_p_opt-1))  
  
    def objective(z):  
        return max((pt - z) * demand_probability[pt] for pt in p) + z * c  
    z_opt = minimize_scalar(objective, bounds=(0, max(p)), method='bounded')  
    return z+((z_opt.x) *(10**5)*(magnitude_order_p_opt))
```

Code 1 : Dual Variable

4.2 Demand Generation:

Demand generation is a central element of our implementation and directly determines whether the primal–dual learning algorithm is tested under conditions that resemble real markets. By “realistic” we mean three things: demand must be price-biased (lower prices generally lead to higher purchase probability), demand must be stochastic (no price should produce deterministic 0% or 100% purchase outcomes), and demand must allow heterogeneity across customer types (different types respond differently to price). To meet these requirements we implement a simple, interpretable logistic acceptance model (shown in the figure) and add a small amount of randomness so that outcomes are never fully deterministic.

The particular functional form we use for the price-to-demand mapping follows the logistic curve shown in the supplied image. In words, this function produces an S-shaped response: around a central reference price the purchase probability changes most

rapidly, and away from that center probabilities saturate toward a lower and an upper level. The curve is controlled by three intuitive parameters. The first parameter sets the overall maximum level of the curve so that the acceptance probability never reaches 100%. The second parameter controls how steeply probability changes with price (the slope or price sensitivity). The third parameter sets the center or “midpoint” of the curve, i.e., the price at which the probability is around the middle of its range. In our implementation we choose these parameters so that the mapping is decreasing in posted price: as the offered price increases, the acceptance probability falls. The S-shaped logistic makes this monotone relationship smooth and easy to reason about.

We explicitly model customer heterogeneity by using different slope and center parameters for each customer type. In the notebook we use two canonical types to illustrate this effect: a reserved type and a preemptive type. Reserved customers are modeled with a shallow slope and a relatively high midpoint, which captures the idea that they are less price sensitive and willing to pay more for certainty or additional features. Preemptive customers are modeled with a steep slope and a lower midpoint, reflecting stronger sensitivity to price changes and a lower typical willingness-to-pay. By varying the slope parameter between types we generate demand curves with markedly different shapes, which is important for testing that the algorithm adapts appropriately after the dual (shadow) price is estimated.

To turn the probability coming from the logistic function into realized sales we sample at the customer-arrival level. For each arriving customer we compute the purchase probability at the posted price and then draw a Bernoulli trial to decide whether that customer buys. For experiments that model mini-batches or high-traffic periods we aggregate these draws (Binomial sampling or Poisson-mixed draws can be used) but the sequential Bernoulli view is the default because it matches the online, one-by-one arrival framework of the primal–dual algorithm.

We add a small, controlled noise term to the logistic probability to reflect unobserved heterogeneity and idiosyncratic decision noise. The perturbation is deliberately modest: its role is to make the outcome mildly random so that learning signals are realistic while preserving the underlying price dependence encoded by the logistic curve. After adding noise we clip the resulting probability to the valid range so sampling remains well defined. We do not treat the noise as the main modeling element; it is only an augmentation to emulate the fact that real consumers do not all behave identically even at the same price.

Practical calibration and validation are important. We pick midpoints and slopes to reflect plausible willingness-to-pay and sensitivity differences across types, and we set the logistic maximum below one so that even the most attractive prices do not imply certain purchase. Typical choices for the slope produce smooth curves rather than step-like behavior; noise variance is kept low so that it does not swamp the price signal. All parameter choices and random seeds are logged so experiments are reproducible. Before running large experiments we always visualize the acceptance curves and run pilot simulations at a handful of prices to confirm that empirical purchase rates match the intended behavior.

Finally, the demand generator is implemented as a compact, modular routine in the notebook. It accepts a price and a small parameter dictionary for a customer type and returns the realized sale outcome for that arrival. This modular design lets us swap the logistic for alternative parametric or nonparametric models if needed, change the noise model, or move from single arrivals to batched arrivals without touching the rest of the algorithm. In short, the logistic-plus-noise construction gives a transparent, controllable, and realistic demand model that supports meaningful evaluation of the primal-dual learning method.

4.2.1 Logistic function:

$$d(p) = \frac{L}{1 + e^{-k(p-p_0)}}$$

Where:

D (p) = demand probability

L = max demand, (here, 1)

k = steepness,

p0= a price in price interval

p = average price

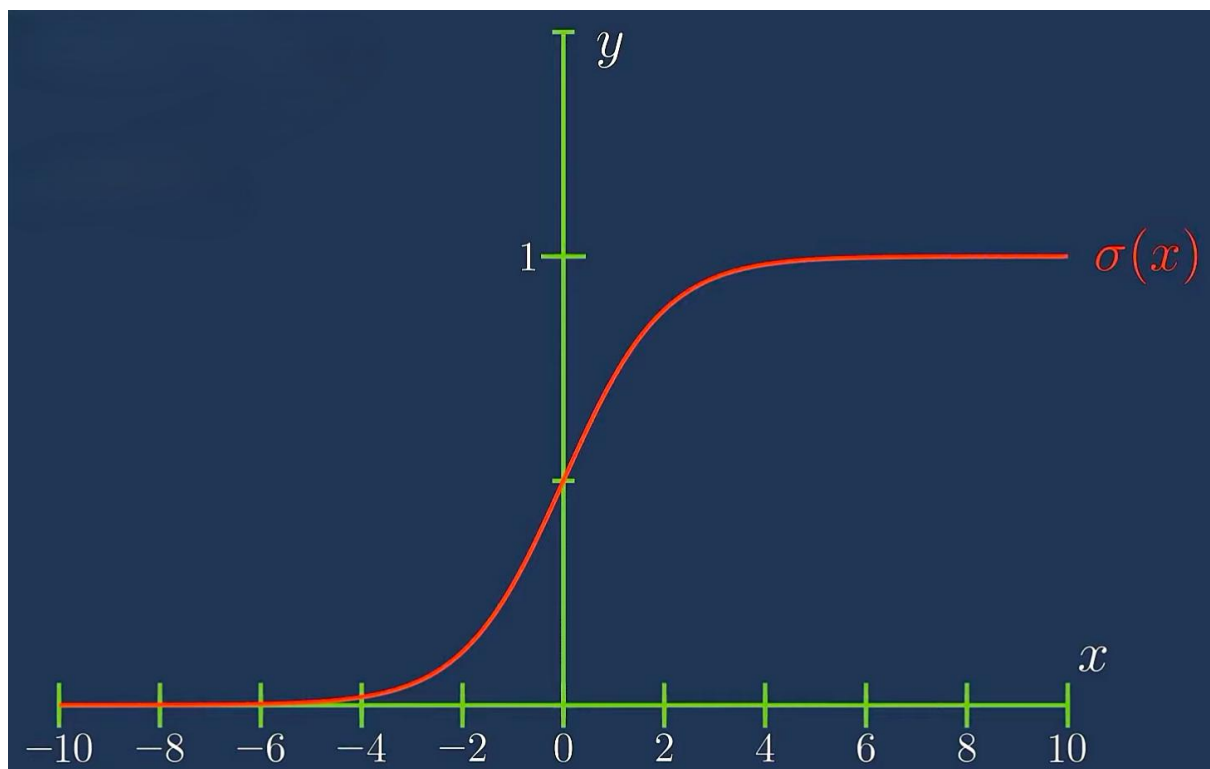


Figure 2: logistic S shaped curve, for representation of logistic function.

The logistic function is a smooth, S-shaped curve widely used to model probabilities that vary between two bounds (typically 0 and 1) as a function of some input, such as price. In our context, it translates the offered price into a purchase probability in a way that captures diminishing sensitivity at extreme prices and the most significant changes around a midpoint.

At low prices, the function approaches its upper bound, meaning customers are very likely to buy, but never with 100% certainty. As prices increase, the probability declines smoothly, with the steepest drop occurring near the midpoint parameter. The slope parameter controls how sensitive demand is to price changes: a steeper slope means even small price changes cause large shifts in purchase probability, while a gentler slope reflects less price sensitivity.

This flexibility makes the logistic function ideal for simulating realistic, price-biased demand while keeping the model interpretable and easy to calibrate.

Code:

```
#####  
#####  
#####  
#""" Demand Function - Returns the base probability of purchase at price p """  
  
# (Logistic) - Demand function - type 1: Reserve  
def logistic_demand_reserved(pt, p, p0=0, L=1, k=0.005 ):  
    p0=(p.min()+p.max())/2  
    return L / (1 + np.exp(k * (pt - p0)))  
  
# (Logistic) - Demand function - type 2: Preemptive  
def logistic_demand_preemptive(pt, p, p0=0, L=1, k=0.04 ):  
    p0=(p.min()+p.max())/2  
    return L / (1 + np.exp(k * (pt - p0)))
```

Code 2: Demand function

4.3 Demand estimation function:

Accurate demand estimation is a cornerstone of dynamic pricing algorithms, particularly in environments where customer behavior is uncertain and inventory is constrained. In the context of telco RAN compute monetization, where GPU capacity is sold to external customers under time-varying conditions, understanding how demand responds to price is essential for making informed allocation and pricing decisions.

4.3.1 Importance of Demand Estimation

The Primal–Dual Learning Algorithm relies on demand estimates to:

- **Project future demand** at a given price point.
- **Compare projected demand with remaining inventory** to determine whether pricing adjustments are necessary.
- **Guide exploitation-phase decisions**, such as whether to continue offering the current optimal price or shift to a higher or lower price to manage inventory consumption.

Without reliable demand estimation, the algorithm risks either underpricing (leading to premature inventory depletion) or overpricing (resulting in lost revenue opportunities). Therefore, the demand estimation function plays a critical role in balancing yield and resource preservation.

4.3.2 Functional Role in the Algorithm

During **Phase II (Exploitation)**, the algorithm takes the optimal price identified in **Phase I (Exploration)** and uses the demand estimation function to:

1. Estimate the probability that a customer will purchase at that price.
2. Multiply this probability by the number of customers approached per day to calculate daily demand.
3. Multiply daily demand by the number of remaining days in the booking horizon to project total future demand.

4. Compare projected demand with remaining inventory to decide whether the current price is sustainable or needs adjustment.

This process is repeated iteratively, allowing the algorithm to adapt to changing demand conditions and inventory levels.

4.3.3 Implementation via Random Forests

In your implementation, demand estimation is performed using a **Random Forest classifier**, a non-parametric ensemble learning method that is well-suited to modeling complex, nonlinear relationships between price and customer behavior. This choice is motivated by two key factors:

- **Unknown demand structure:** The relationship between price and demand is not assumed to follow a specific functional form (e.g., linear or logistic). Instead, demand is generated with noise and bias to simulate realistic market conditions, including price sensitivity and randomness.
- **Robustness to noise and bias:** Random Forests are capable of handling noisy data and capturing subtle patterns, making them ideal for learning from biased or irregular demand signals.

4.3.4 Code Functionality

The demand estimation function, `random_forest_buying_probability`, performs the following steps:

1. **Data Preparation:**

The function takes a filtered DataFrame containing customer interactions and extracts the relevant features — in this case, price and purchase outcome.

2. **Model Training:**

The data is split into training and test sets. A grid search is performed over a range of hyperparameters (number of trees, depth, leaf size, feature selection

strategy) using 5-fold cross-validation to identify the best-performing Random Forest model.

3. **Model Evaluation:**

The selected model is evaluated on the test set to ensure it generalizes well. Accuracy is computed to validate performance.

4. **Probability Prediction:**

The trained model is used to predict the probability that a customer will purchase at a given price. This probability is returned as the estimated demand rate for that price.

This function is called during the exploitation phase to estimate demand at the current optimal price, enabling the algorithm to project future demand and make inventory-aware pricing decisions.

4.3.5 Strategic Value

By using Random Forests for demand estimation, your algorithm gains:

- **Flexibility** in modeling diverse customer behaviors.
- **Resilience** to noise and bias in simulated demand data.
- **Scalability** across different product types and pricing intervals.

This approach ensures that pricing decisions are grounded in data-driven insights, even when the underlying demand environment is complex and uncertain.

4.3.6 Random Forests for Demand Estimation

Random Forests are a powerful ensemble learning method used for both classification and regression tasks. In the context of demand estimation, they offer a flexible and robust way to model the relationship between price and customer purchasing behavior — especially when that relationship is nonlinear, noisy, or influenced by hidden variables.

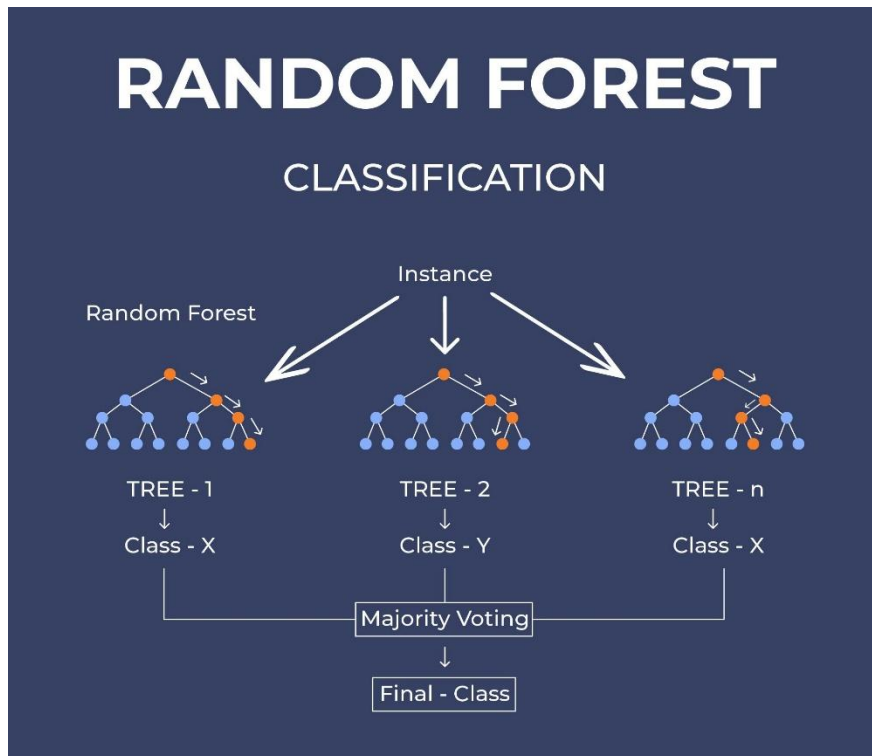


Figure 3: Random forest representation image.

What Is a Random Forest?

A Random Forest is essentially a collection (or "forest") of decision trees. Each tree is trained on a random subset of the data and features, and the final prediction is made by aggregating the outputs of all trees — typically by majority vote (for classification) or averaging (for regression).

Key Concepts:

- **Decision Tree:** A flowchart-like structure that splits data based on feature values to make predictions.
- **Bootstrap Aggregation (Bagging):** Each tree is trained on a random sample of the data with replacement, which reduces variance.

- **Feature Randomness:** At each split, only a random subset of features is considered, which decorrelates trees and improves generalization.
- **Ensemble Averaging:** The final prediction is the average (for probabilities) or majority vote across all trees, making the model more stable and accurate.

Why Random Forests Are Ideal for Demand Estimation

Your demand generation process introduces **bias and randomness** to simulate real-world customer behavior. This makes traditional parametric models (like linear regression or logistic regression) less effective, as they assume a fixed functional form. Random Forests, by contrast, offer:

- **Nonlinear Modeling:** They can capture complex, nonlinear relationships between price and buying probability.
- **Noise Tolerance:** Random Forests are robust to noisy data and overfitting, thanks to ensemble averaging.
- **No Assumptions About Data Distribution:** They don't require the data to follow any specific distribution, which is ideal for synthetic or biased demand signals.
- **Feature Importance Insights:** They can reveal which features (e.g., price, customer type) most influence buying behavior.

How It's implemented

In your implementation:

- You train a Random Forest classifier on historical customer data, where the input is **price** and the target is **buying behavior** (binary: buy or not).
- You use **GridSearchCV** to tune hyperparameters like the number of trees (`n_estimators`), tree depth (`max_depth`), and feature selection strategy (`max_features`).

- Once trained, the model predicts the **probability** that a customer will buy at a given price — this becomes your demand estimate.

Example: Predicting Buying Probability

Suppose your optimal price from Phase I is €120. The trained Random Forest might predict a buying probability of 0.63, meaning 63% of customers are expected to purchase at that price. This probability is then used to forecast future demand and guide inventory-aware pricing decisions.

Code:

```
#####
#####
#####
# Calculate buying probability based on demand generated in exploration

def random_forest_buying_probability(df, price_value):
    """
    Trains a Random Forest classifier and predicts buying probability for a
    given price.
    Prints model accuracy on training data.

    Parameters:
    df (DataFrame): Filtered DataFrame for customer_type == 1
    price_value (int or float): Target price for prediction

    Returns:
    float: Predicted probability of buying
    """
    df_clean = df.copy()
    X = df_clean[['price']]
    y = df_clean['buy'].astype(int)

    # Split your data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

    # Define hyperparameter grid
    param_grid = {
        'n_estimators': [100, 200, 500],
        'max_depth': [None, 5, 10],
```

```

        'min_samples_leaf': [1, 2, 4],
        'max_features': ['sqrt', 'log2']
    }

    # Set up GridSearchCV
    grid_search = GridSearchCV(
        RandomForestClassifier(random_state=42),
        param_grid,
        cv=5,                      # 5-fold cross-validation
        scoring='accuracy',
        n_jobs=-1
    )

    grid_search.fit(X_train, y_train)
    best_model = grid_search.best_estimator_

    # Evaluate accuracy on test set
    test_predictions = best_model.predict(X_test)
    test_acc = accuracy_score(y_test, test_predictions)
    # print(f"Tuned Random Forest test accuracy: {round(test_acc, 2)}")
    # print("Best parameters found:", grid_search.best_params_)

    # Predict probability for input price
    predicted_prob = best_model.predict_proba([[price_value]])[0][1]
    return round(predicted_prob, 2)

```

Code 3 : Random Forest function (random_forest_buying_probability) for demand estimation

5. Results

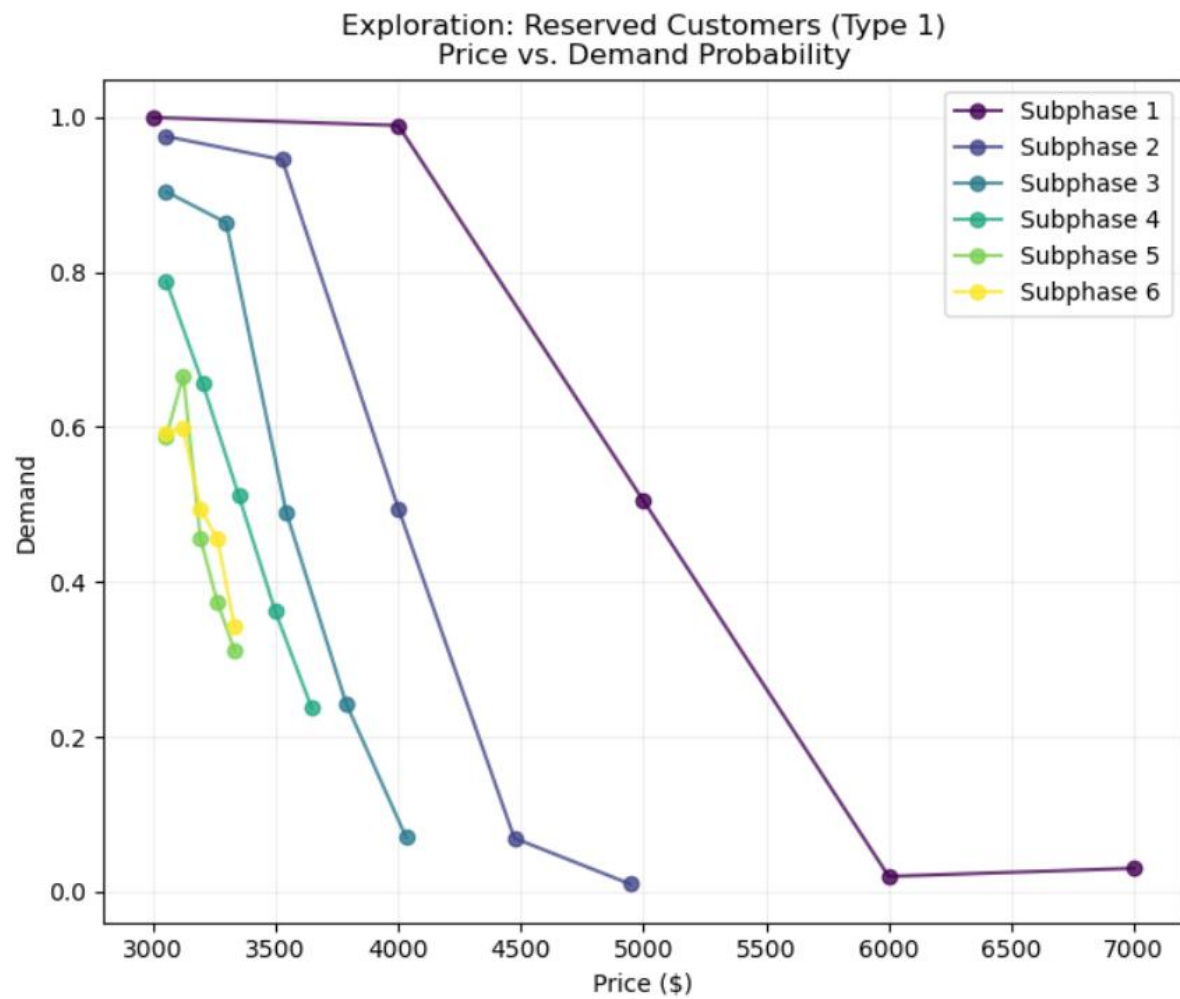


Figure 4: Price vs demand in phase 1 for customer type 1

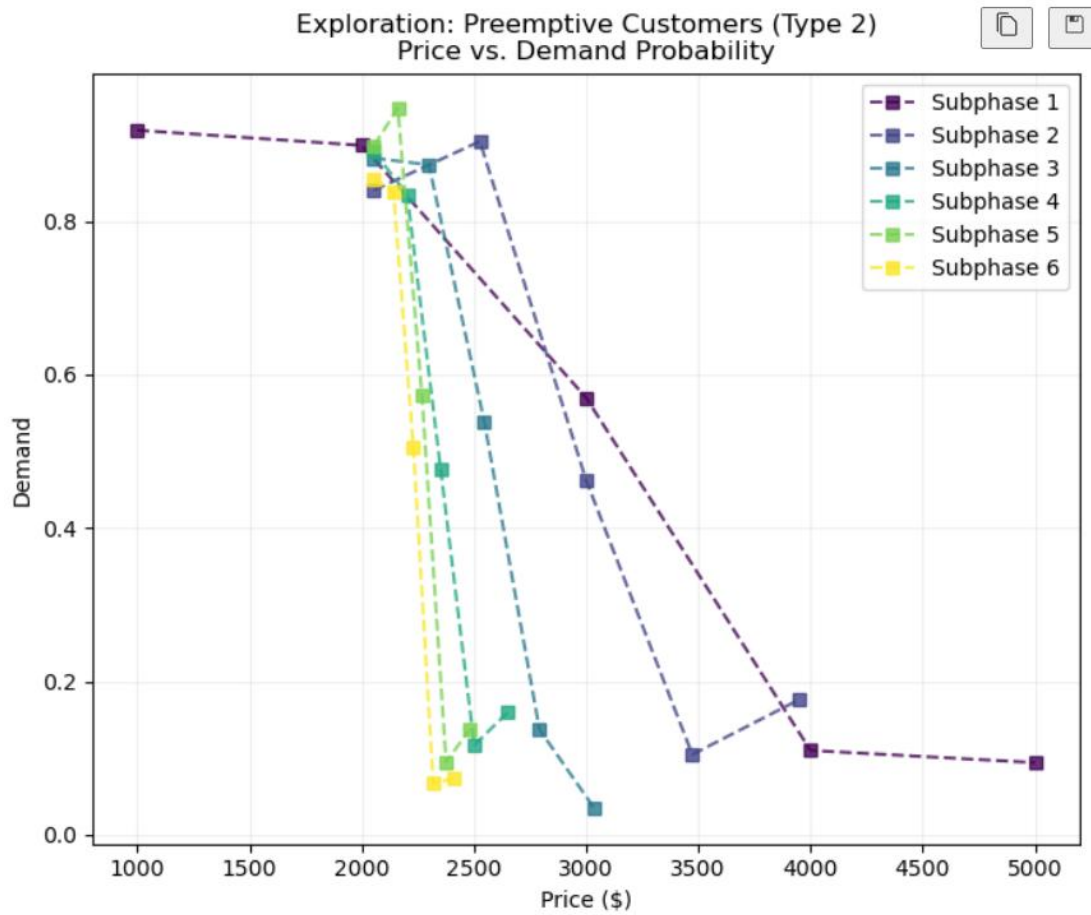


Figure 5: Price vs demand in phase 1 for customer type 2

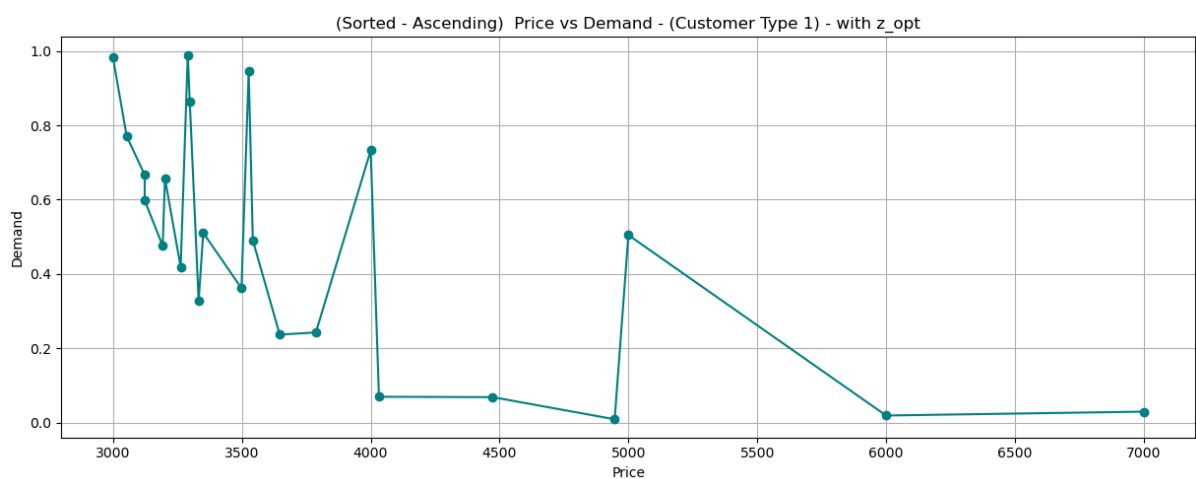


Figure 6: Price vs demand after all phases for customer type 1 – considering z_{opt} means considering absolute revenue

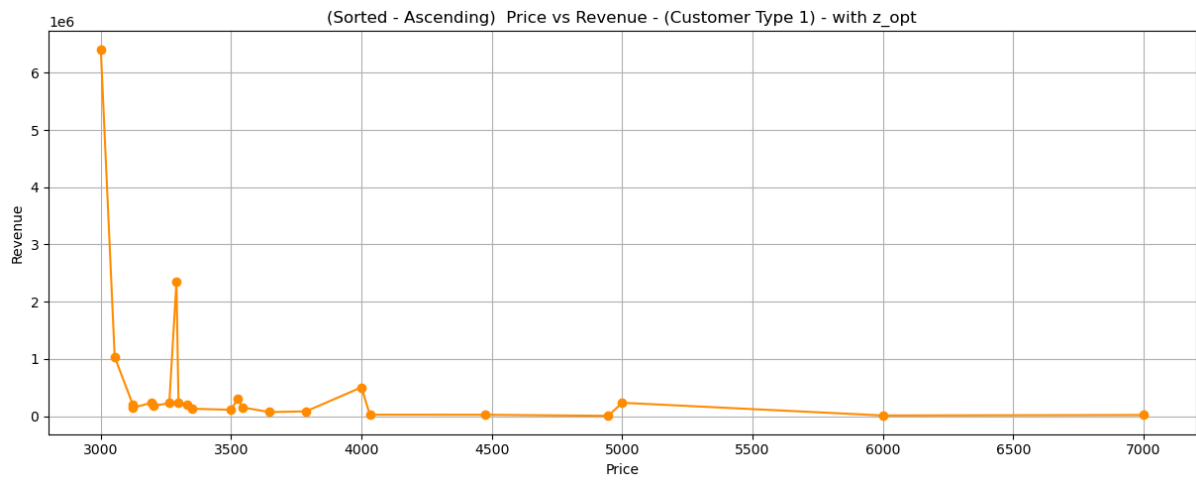


Figure 7: Price vs revenue after all phases for customer type 1 – considering z_{opt} means considering absolute revenue

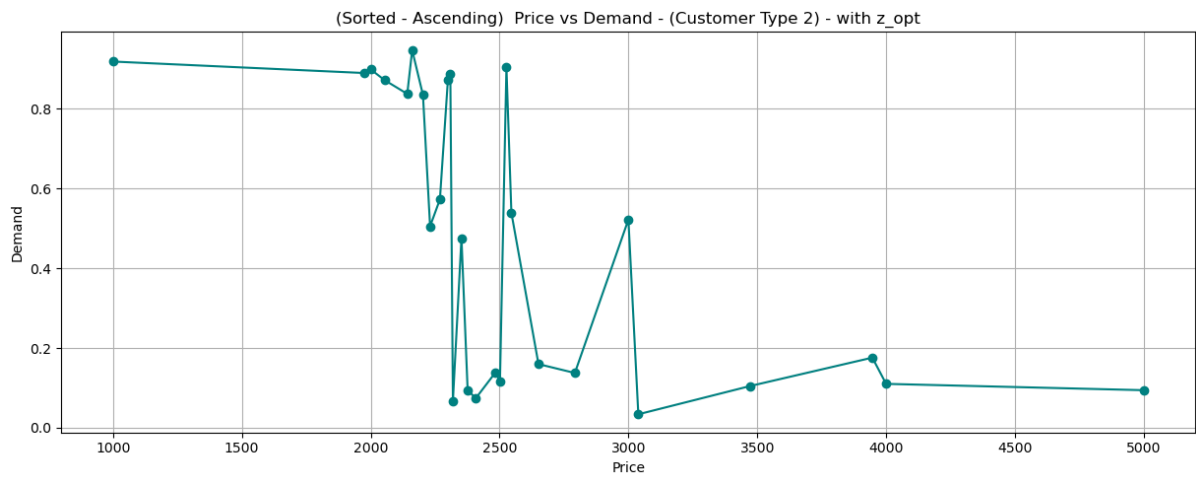


Figure 8: Price vs demand after all phases for customer type 2 – considering z_{opt} means considering absolute revenue

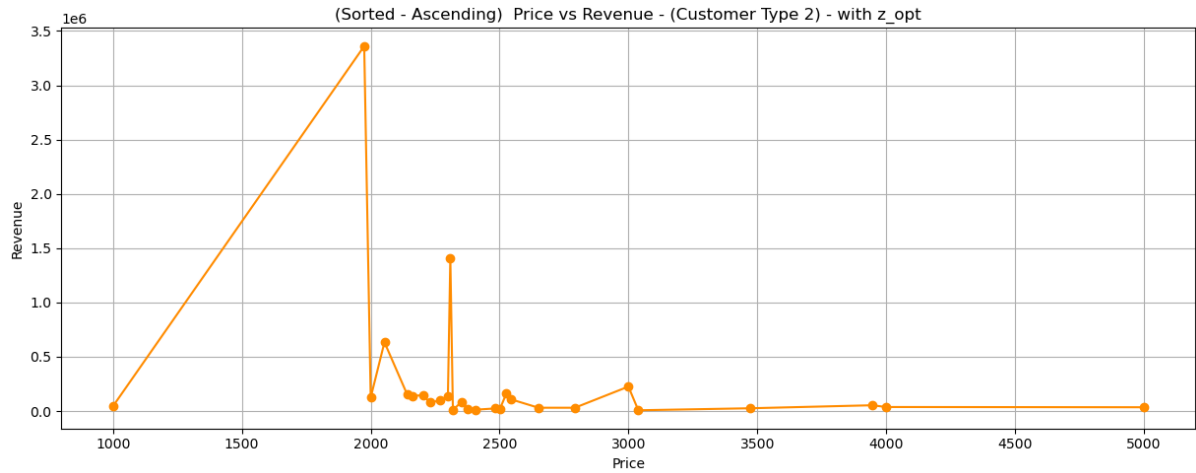


Figure 9: Price vs revenue after all phases for customer type 2 – considering z_{opt} means considering absolute revenue

5.1 Observations and Performance Analysis

The performance of the pricing algorithm was evaluated across multiple dimensions, including price interval convergence, regret analysis, and the relationship between price, demand, and revenue. These observations provide critical insights into the behavior of the algorithm and its effectiveness in learning optimal pricing strategies under uncertainty.

5.1.1 Convergence of Price Intervals in Phase 1

During the exploration phase, the algorithm iteratively refines its understanding of customer demand by offering prices across a predefined range. A key observation is that the **price intervals progressively narrow** with each iteration. This narrowing reflects the algorithm's increasing confidence in identifying the region around the optimal price.

- Initially, prices are sampled broadly to capture demand behavior across the entire feasible range.
- As more data is collected, the algorithm begins to concentrate its sampling around prices that yield higher adjusted revenue.

- This convergence behavior is a hallmark of efficient exploration, indicating that the algorithm is successfully learning from customer responses and adapting its strategy accordingly.

The narrowing of price intervals is particularly evident in customer type 2, where the algorithm quickly homes in on the optimal price region. In contrast, customer type 1 exhibits slower convergence, suggesting more variability or noise in demand responses for this segment.

5.1.2 Regret Analysis Across Customer Types

Regret is a key metric in online learning algorithms, representing the difference between the revenue achieved by the algorithm and the revenue that would have been obtained had the optimal price been known from the outset.

- For **customer type 2**, the observed regret ranges between **5% and 13%**, which is well within the acceptable threshold of 0% to 15%. This indicates that the algorithm performs efficiently for this segment, learning the optimal pricing strategy with minimal loss.
- For **customer type 1**, regret ranges between **9% and 18%**, with some instances exceeding the 15% threshold. This suggests that the algorithm faces greater challenges in accurately estimating demand or adjusting prices for this customer type.

The higher regret in customer type 1 may be attributed to several factors:

- Greater heterogeneity in demand behavior.
- Lower sensitivity to price changes, making it harder to distinguish optimal pricing points.
- Potential overlap in demand probabilities across multiple prices, leading to slower convergence.

These findings underscore the importance of tailoring exploration strategies to different customer segments, especially when demand patterns vary significantly.

5.1.3 Price vs. Demand and Price vs. Revenue Relationships

A critical insight from the empirical analysis is the **nonlinear relationship between price, demand, and revenue**. While high demand is often associated with favorable pricing, it does not necessarily translate into high revenue.

- In the **price vs. demand plots**, demand peaks at multiple price points. This suggests that customers may be willing to purchase at both low and moderately high prices, depending on their valuation and urgency.
- However, the **price vs. revenue plots** reveal that not all high-demand prices yield high revenue. This discrepancy arises because revenue is a product of both price

and demand — a high demand at a low price may result in suboptimal revenue, whereas moderate demand at a higher price can be more profitable.

This observation highlights the importance of **adjusted revenue** as the guiding metric in the algorithm. By incorporating the dual variable, the algorithm accounts not only for immediate revenue but also for the opportunity cost of consuming inventory. As a result, it avoids the pitfall of chasing high demand at the expense of long-term profitability.

5.1.4 Strategic Implications

The empirical results validate the core design principles of the algorithm:

- **Adaptive learning:** The narrowing of price intervals demonstrates that the algorithm effectively learns from customer behavior.
- **Segment-specific performance:** Regret analysis reveals that different customer types require different exploration intensities.
- **Revenue-centric optimization:** The divergence between demand and revenue underscores the need for economically grounded pricing strategies.

These insights provide a strong foundation for refining the algorithm further, potentially by incorporating segment-specific exploration rates, dynamic regret thresholds, or more granular demand modeling techniques.

6. Abbreviations:

List of Symbols and Abbreviations

Symbol	Description
p	Price of a product or service
p_{low}	Lower bound of the price search interval
p_{upp}	Upper bound of the price search interval
p^*	True optimal price under full-information demand
M	Number of products or customer types
m	Product index (example, Reserved = 1, Preemptive = 2)
T	Total selling horizon (in discrete time periods)
t	Discrete time period index
K	Number of explorations subphases
c_0	Initial inventory (capacity)
c	Remaining inventory at a given time
$d_m(p)$	Demand function for product m at price p
β	Price sensitivity parameter
$P_m(p)$	Probability of purchase for product m at price p
z	Dual variable (shadow price of inventory)
z^*	Optimal dual variable
π	Pricing policy
δ	Price interval narrowing factor
R	Revenue function
Regret	Difference between benchmark and achieved revenue
K	Number of exploration subphases in Phase 1
$J_D(T, c)$	Benchmark expected revenue under deterministic full-information demand
$J_\pi(T, c)$	Expected revenue under policy π
θ	Parameter vector for the demand function
ε	Error term or noise in demand estimation
η	Learning rate (when applicable)
q_m	Sales quantity for product m
$\tilde{P}$	Adjusted or candidate price in narrowing steps
λ	Poisson demand rate parameter
λ_m	Poisson demand rate for product m

7. References

Gallego, G., & van Ryzin, G. (1994). Optimal Dynamic Pricing of Inventories.

Chen, N., & Gallego, G. (2018). A Primal-Dual Learning Algorithm for Personalized Dynamic Pricing.

Besbes, O., & Zeevi, A. (2009). Dynamic Pricing Without Knowing the Demand Function.

Ferreira, K. J., et al. (2018). Deep Learning for Revenue Optimization in E-Commerce.

Javanmard, A., & Nazerzadeh, H. (2019). Dynamic Pricing in High-Dimensions.